

Real-Time 3D Hourglass Simulation Report

โครงการ: Realtime Computer Graphics, Physics Simulation Final Project

ชื่อผู้ทำโครงการ: วินิทรามโนเลิศเทเวศ

รหัสนิสิต: 6530375021

อาจารย์ที่ปรึกษา: รศ. ดร. ณัฐพงศ์ ชินธเนศ

รายวิชา: 2110514 REALTIME COMPUTER GRAPHICS AND PHYSICS SIMULATION

ภาคการศึกษา: ภาคปลาย ปีการศึกษา 2569

GitHub: <https://github.com/winithraKS/realtime-cg-final-project.git>

Youtube Demo: <https://youtu.be/KX2f0Z5ALGI>

1. Overview

This project implements a 3D hourglass simulation using Python, ModernGL, NumPy, and Pygame. It utilizes Verlet integration and position-based dynamics to model granular sand motion within a transparent container. To simplify collision handling, the engine evaluates gravity in local space instead of transforming the physical container geometry directly.

Main features include:

- Real-time 3D particle simulation with variable radii
- Spatial hash optimization for efficient collision detection
- Interactive camera controls and smooth flipping animations

2. System Architecture

The simulation pipeline executes sequentially every frame, updating particle physics before rendering:

System	Responsibility
Particle System	Simulates granular physics, movement, and boundaries
Spatial Hash	Accelerates neighbor searching via uniform grid mapping
Collision System	Resolves particle-particle and particle-wall overlaps
Mesh Generator	Generates procedural sphere and hourglass geometries
Rendering System	Draws instanced particles and transparent glass shell
Camera & Input	Processes interaction (orbit, zoom, pan, flip, reset)
HUD Renderer	Displays real-time FPS and simulation statistics

3. Particle System

Each particle stores its current position, previous position, velocity, and radius. The simulation operates in a local space where the hourglass remains mathematically upright.

3.1 Particle Initialization

Particles spawn randomly inside the upper chamber with randomized radii to prevent artificial grid crystallization:

$$r_i \in [0.9r, 1.1r]$$

Overlap testing ensures a collision-free spawning state:

$$d > r_i + r_j$$

where d represents the Euclidean distance between two particle centers.

3.2 Simulation Pipeline

1) Local Gravity Transformation

To simulate flipping without transforming static collision geometry, gravity is rotated into local space using the inverse Z-axis rotation matrix R^{-1} :

$$g_{\text{local}} = R^{-1} \cdot g_{\text{world}}$$

2) Verlet Integration

Positions update via Verlet integration, deriving implicit velocity from historical positions to ensure granular stability:

$$x_{\text{new}} = x + (x - x_{\text{prev}}) + g\Delta t^2$$

3) Spatial Hash and Neighbor Collision

The domain is partitioned into uniform grid cells. Particles only evaluate pairs within adjacent cells. Overlaps ($d < r_i + r_j$) are resolved by projecting particles apart along the contact normal n :

$$n = \frac{p_i - p_j}{|p_i - p_j|}$$
$$\text{correction} = n \cdot \frac{\text{overlap} - \text{slop}}{2}$$

Tangential relative velocity is damped to simulate friction and allow realistic sand piles to form.

4) Boundary Constraints (Hourglass Clamping)

Particles are bound within the container using linear interpolation based on height y :

$$r(y) = r_{\text{neck}} + t(r_{\text{top}} - r_{\text{neck}}), \quad t = \frac{y - y_{\text{neck}}}{y_{\text{top}} - y_{\text{neck}}}$$

To prevent wall penetration along slanted surfaces, the boundary limit accounts for the cone slope angle θ :

$$r_{\text{limit}} = r(y) - \frac{r_{\text{particle}}}{\sin(\theta)}$$

5) Velocity Reconstruction and Damping

Post-collision velocities are reconstructed and slightly damped to dissipate excess numerical energy:

$$v = \frac{x - x_{\text{prev}}}{\Delta t}$$

6) Verlet State Resynchronization

Previous positions are updated to maintain integration consistency and prevent numerical drift:

$$x_{\text{prev}} = x - v\Delta t$$

3.3 Reset Function

Clears active velocities and re-runs the initial spawn sequence in the upper chamber, enabling instantaneous simulation resets.

4. Procedural Mesh Generation

4.1 Hourglass Geometry

The container shell consists of an upper cone, a lower cone, and top/bottom flat discs generated using cylindrical coordinates:

$$x = r \cos(\theta), \quad z = r \sin(\theta)$$

Vertex normals are derived directly from the cone slope vectors to ensure proper lighting and smooth Phong shading specular highlights.

4.2 Sand Particle Geometry

A base sphere mesh is generated via standard latitude-longitude tessellation:

$$x = r \cos(\phi) \cos(\theta), \quad y = r \sin(\phi), \quad z = r \cos(\phi) \sin(\theta)$$

To minimize draw call overhead, the engine implements **GPU Instanced Rendering**. The sphere topology is loaded into VRAM once, while per-particle positions, speeds, and radii are streamed into instance attributes every frame.

5. Render Pipeline

Execution order follows physics completion:

1. Update instance buffers with active world positions and attributes.
2. Render sand particles via instanced arrays.
3. Render transparent hourglass shell using a two-pass technique
4. Render 2D HUD text overlay onto a fullscreen quad.

The visual rotation matrix $R_z(\theta)$ is applied only to the rendered meshes, keeping the physics engine decoupled and static.

6. Input System

Input	Function
Left Mouse Drag	Orbit camera (Yaw/Pitch)
Shift + Drag	Pan camera target (XY)
Scroll Wheel	Zoom camera distance
F Key	Flip hourglass 180°
R Key	Reset particle simulation
Space	Pause / Resume simulation
ESC	Exit application

The flip interaction increments a target angle ($\theta_{\text{target}} + = \pi$), which the visual rotation matrix smoothly interpolates toward over time.

7. Conclusion

This project implements a real-time 3D granular simulation using position-based dynamics. Local-space gravity evaluation maintains stability without complex collision geometry updates, while spatial hashing effectively reduces neighbor-search complexity.

8. Special Thanks

- **Claude:** Provided foundational starter code blocks for the simulation pipeline architecture.
- **Gemini:** Assisted in refining the hourglass boundary constraints and proofreading this report.
- **ChatGPT:** Fostered the structure and initial drafting of the technical report documentation.